单选 T0015-1-1-1 ★ 回答错误

阅读下列程序，给出正确的结果：

```
class Bee{
    int honeyBag = 2;
    public void pickHoney(){
        honeyBag ++;
    }
}

class Test extends Bee{
    int honeyBag = 4;
    public void pickHoney(){
        honeyBag += 5;
    }

    public static void main(String[] args) {
        Bee b = new Test();
        b.pickHoney();
        System.out.println(b.honeyBag);
    }
}
```

你的答案

Ⓐ 4

Ⓑ 9 ✕ 错误

Ⓒ 3

Ⓓ 2 ✓ 正确

单选 T0015-1-3-1 ★ 回答正确

阅读下列程序，给出正确的结果：

```
class Bee{
    int honeyBag = 2;
    public void pickHoney(){
        honeyBag ++;
    }
}

class Test extends Bee{
    int honeyBag = 4;

    public static void main(String[] args) {
        Bee b = new Test();
        b.pickHoney();
        System.out.println(b.honeyBag);
    }
}
```

你的答案

Ⓐ 7

Ⓑ 9

Ⓒ 3 ✓ 正确

Ⓓ 2

阅读下列程序，给出正确的结果：

```
class Bee{
    int honeyBag = 2;
    public void pickHoney(){
        honeyBag ++;
    }
}

class Test extends Bee{
    public void pickHoney(){
        honeyBag += 5;
    }

    public static void main(String[] args) {
        Bee b = new Test();
        b.pickHoney();
        System.out.println(b.honeyBag);
    }
}
```

你的答案

Ⓐ 4

Ⓑ 7 ✓ 正确

Ⓒ 3

Ⓓ 2

对于问题 1，执行 b.pickHoney ();时，由于 b 实际上是一个 Test 对象（尽管它的引用类型是 Bee），因此会调用 Test 类的 fly 方法。这个方法将 Test 类中的 a 字段增加 5，使其值变为 9。然而，执行 System.out.println(b.honeybag);时，这里发生了一个不同的事情。由于 b 的编译时类型是 Bee，Java 会尝试访问 Bee 类中的 a 字段。尽管 b 实际上指向一个 Test 对象，但由于字段隐藏的规则，不能通过 Bee 类型的引用来直接访问 Test 类中的 a 字段。因此，这行代码将输出 Bee 类中的 a 字段的值，即 2（因为 Bee 类的 fly 方法没有被调用，所以它的 a 字段值没有改变）。

对于问题 2，在您的代码中，Test 类没有重写 Bee 类的 fly 方法。因此，当您通过 Bee 类型的引用 b（该引用实际上指向一个 Test 对象）调用 fly 方法时，调用的是 Bee 类的 fly 方法。这个方法将 Bee 类中的 a 字段增加了 1。接着，当您尝试通过相同的 Bee 类型的引用 b 访问 a 字段时，由于字段隐藏的规则，您访问的是 Bee 类中的 a 字段，而不是 Honee 类中隐藏的 a 字段。因此，System.out.println(b.a);输出的是 Bee 类中的 a 字段的值，该值由于 fly 方法的调用已经从 2 增加到了 3。所以，最终输出是 3。

对于问题 3，Bee b = new Test (); 创建了一个 Test 对象，并用一个 Bee 类型的引用 b 来指向它。b. pickHoney (); 调用的是 Test 类的 pickHoney 方法，因为 b 实际上指向一个 Test 对象。这个方法将 a 字段的值从 2 增加到 7。System.out.println(b.a); 访问的是 Test 对象的 a 字段，因为 b 指向的是 Test 对象。由于 Test 的 fly 方法已经修改了 a 的值，所以输出的是 7。问题 3 相对于 1 和 2，在 Test 类中没有对 a 进行申明，所以不存在字段隐藏规则，访问的是 Test 类中的 a 字段，所以最后打印的是 Test 对象的 a 字段 7，但需要知道 Bee 对象的 a 字段仍为 2。
过程：**类定义**：Bee 类有一个名为 a 的字段，初始值为 4。
Bee 类有一个 fly 方法，该方法将 a 字段的值增加 1。
Test 类继承了 Bee 类。
Test 类重写了 fly 方法，该方法将 a 字段的值增加 5（注意，这里使用的是 Test 继承自 Bee 的 a 字段，而不是在 Honee 中定义的一个新字段，因为 Honee 并没有声明一个新的 a 字段）。
main 方法：在 main 方法中，创建了一个 Bee 类型的引用 b，但它指向了一个 Test 对象。
调用 b. pickHoney ()时，由于 b 实际上指向一个 Test 对象，因此会调用 Test 类中的 pickHoney 方法。
Test 的 fly 方法将 a 字段的值从 2 增加到 7（2 + 5 = 7）。
输出：System.out.println(b.a);语句输出的是 Bee 类中定义的 a 字段的值。
由于 b 虽然是一个 Bee 类型的引用，但它指向的是 Test 的实例，并且 Test 没有隐藏（即没有定义一个新的同名字段）Bee 类中的 a 字段，因此这里访问的是同一个 a 字段。
由于 Test 的 pickHoney 方法已经将 a 字段的值增加到 7，因此输出将是 7。

问题 4

```
class Bee {
   public void fly() {
      a++;  } }
class Honee extends Bee {
   int a=4;
   // 重写 fly 方法（可选，但展示了多态性）
   @Override
   public void fly() {
      a+=5 ;  }
   public static void main(String[ ] args) {
      Bee b=new Honee();
      b.fly();
      System.out.println(a); }
}
```

a 的作用域：在 Bee 类中没有定义 a，而在 Honee 类中定义了 a。在 Bee 类型的引用 b 中无法直接访问 Honee 的 a 变量。
多态与方法重写：通过 Bee b = new Honee(); 创建的对象调用 b.fly(); 时，会执行 Honee 类中的 fly() 方法，因为方法是动态绑定的。
输出结果：System.out.println(b.a); 中的 a 访问了 Bee 类的 a，而 Bee 类并没有定义 a，这会导致编译错误。

1. 下面哪些说法是错误的，A,B,C
   A. 环境变量 path 的设置是 java 语言的范畴
   B. 执行一个 Java 程序的指令是：java xxxx.class
   C. Java 语言是区分大小写的，所以对 HelloWorld 类的编译语句不能写成
      javac helloworld.java
   D. Java 程序中 public 的类名要跟该程序文件名同名

   原因：A. 环境变量 PATH 的设置是操作系统层面的配置，用于指定可执行文件的搜索路径。虽然 Java 的安装和配置（如 JAVA_HOME 和 PATH）对于运行 Java 程序是必要的，但这不属于 Java 语言本身的范畴，而是属于系统配置的一部分。
   B. 没有 class 后缀
   C. 打开命令行窗口，先输入 javac HelloWord.java，再输入 java HelloWorld
   在编写的 java 文件的目录下输入 javac HelloWord.java 不报错误就是编译成功。这里的 HelloWorld 大小写都没关系，因为这里是 Windows 层面上的，**Windows 不区分大小写。但在 java 语句中，区分大小写，要写成** java HelloWorld

2. 一个类 A，在语句"'A a=new A() ;"执行之后，生成的对象存在于内存中的_堆（heap）中,而对象的句柄 a 则存在于内存的_栈(堆栈,stack)_中。

3. Public class Bee00{
   public static void main(String[] args){
   Bee00 bee1= new Bee00(1, 50, 50);
   Bee00 bee2= new Bee00(2, 100, 100);
   bee1=bee2;}
   }
   该语句执行后，在堆中生成了 2 个对象，但另一个对象会被释放掉。bee1 和 bee2 都指向同一个对象，这个对象是 bee2 所指的对象。

4. 下列代码所在程序**不会**遇到编译错误
   class T2 {
   float z;
   public void setValue(int x, float y){
   z= x+y;}
   public void setValue(float x, int y){
   z=x+y;}
   }

5. 下列代码运行的结果是，**4**
   Class Bee{
   int honeyBag =2;
   public void pickHoney(){
   honeyBag++; // 增加父类的 honeyBag 变量的值
   }
   }

```
Class Test extends Bee{
        int honeyBag = 4; // 子类的成员变量，遮蔽了父类的 honeyBag
        public static void main(String  args[]){
            Test b = new  Test();
            b.pickHoney();//在 pickHoney()方法内部，honeyBag 引用的是父类 Bee 中的 honeyBag 变量（因为这是在父类的方法中）。这个方法将父类 Bee 的 honeyBag 变量的值从 2 增加到 3。
            System.out.println(b.honeyBag);// 访问的是子类 Test 中的 honeyBag 变量。由于变量遮蔽，这个变量与父类 Bee 中的 honeyBag 变量是两个不同的变量。子类 Test 的 honeyBag 变量在对象创建时被初始化为 4，并且没有被 pickHoney()方法修改。
        }
}
```

6. 下列代码运行的结果是，**9**

```
Class Bee {
    int honeyBag =2;
    public void pickHoney(){
    honeyBag++;}
}

Class Test extends Bee{
    int honeybag = 4;
    public  void pickHoney(){
        honeybag+=5;}

    public static void main(String [ ]  args){
        Test b= new Test( );
        b.pickHoney( );
        System.out.println(b.honeyBag) ;
    }
}
```

7.下列代码执行时，在 new Honee()对象的时候，堆中是 Honee 的对象，而非 Bee 的对象

```
Class Bee{
int honeyBag = 2 ;
public void pickHoney( ){
    honeybag++ ;  }
}

Class Test extends  Bee{
public void pickHoney( ){
    honeybag +=5 ;   }
}
```

```
public class Test{
    public static void main(String[ ] args) {
        Honey bee = new Honee() ;   }
}
```

1.抽象类可以继承，也可以 new 一个实例。**错误**

2. 抽象类中可以没有抽象方法。

3.final 修饰的类不能继承，但可以 new 一个对象。

4.声明成员变量时，如果不使用任何访问控制符（public, protected, private）,则
不同包中的子类 的类不能对成员进行直接访问。
同一类，同一包中的子类，同一包中的非子类 的类都能对成员进行直接访问

5.被私有访问控制符 private 修饰的成员变量，**只能被该类自身所修改和访问**。

6.表示结束本次循环（不是结束整个当前循环），执行下一次循环的指令是 **continue**

7. 关于下列程序

```
public  class  Condition  {
    public static void main ( String[ ] args) {
        int i = 3;
        if ( i=4 )
            System.out.println ( “i!=3”);
        else
            System.out.println ( “i==3”);
        }
}
```

输出结果是：编译出错。因为 if 判断语句中应该是==号，而不是=。

8. 关于下面代码，不能通过不同类型的返回值进行重载，无法编译通过。

```
Class T2 {
    public int getValue( ) {
        return 24;
        }
    public float getValue() {

        return 24f ; }  }

Public class Ext6{
    Public static void main ( String [ ] args ) {
        {
            T2 t1=new T2( );
            System.out.println(t1.getValue( )) ; }
} }
```

一．封装

1. 定义：将相关的变量跟方法放在一起，跟无关的方法隔离，精确地控制变量的访问。
2. 封装是为了有效描述对象，即将代码有效分隔。作用是隔离和重用。
3. 类是一类主体属性与方法的集合，例如：蜜蜂类。可以通过蜜蜂类创造出这个类的许多个体，比如：蜜蜂甲、蜜蜂乙等，它们是蜜蜂类的具体对象。
4. 内存中的对象跟代码中的类有差异，对象实际存在的只是变量的集合，方法不在对象中。对象的句柄存放在栈 Stack 中，对象本体存放在堆 Heap 中，方法存放在代码区中
5. 小结：主程序重要的是去 new 不同类的对象，让对象自身去互动，不要去干涉对象的行为细节。类是数据结构和操作该数据方法的集合，核心是数据。
6. 在编程中，类（Class）与对象（Object）的区别如下：

   类：
   - 是一种抽象的数据类型，它定义了对象的属性和行为（即方法和属性）。
   - 可以看作是一个模板或蓝图，用于创建具体的对象实例。

   对象：
   - 是类的实例，根据类中的定义，它包含了具体的属性值和行为实现。
   - 对象是类的具体化，每个对象都是类的一个实例，通过对象可以访问类中的属性和方法。

   简而言之，类是对象的模板或定义，而对象是类的具体实现。
7. main 函数的存在是为了：

   提供统一的入口：确保程序从一个明确的点开始执行。

   便于操作系统和 JVM 管理：操作系统和 JVM 知道在程序启动时应该调用哪个函数。

   传递命令行参数：main 函数可以接受命令行参数，这使得程序能够根据用户输入或配置来定制其行为。

8．在 Java 中，**每个类只能有一个 main 方法**，且其签名必须是 public static void main(String[] args)。这是 Java 语言的规范所决定的。

然而，**不同的类可以各自拥有一个 main 方法**。这意味着你可以在一个 Java 项目中定义多个类，每个类都有一个 main 方法。但是，当你运行程序时，你必须指定包含要执行的 main 方法的类的名称。JVM（Java 虚拟机）将只调用该类的 main 方法，而不会调用其他类的 main 方法。在 Java 项目中可以定义多个 main 方法（只要它们位于不同的类中），但在程序的每次执行中，只能有一个 main 方法被调用作为程序的入口点。

9. 对象在内存中是一个包含对象头、实例数据和对齐填充的数据结构，通常被分配在堆上，并通过引用来访问。对象的生命周期包括创建、使用和销毁三个阶段，由编程语言运行时环境自动管理。

10. **this 和 super 关键字**：

this

用途：this 关键字用于引用当前对象。

场景：在实例方法中，this 指向调用该方法的对象。在构造函数中，this 用于调用同一个类的另一个构造函数（必须是构造函数的第一条语句）。

当局部变量与成员变量同名时，使用 this 来区分成员变量和局部变量。

作用：确保对象的属性和方法被正确访问和调用。

super
用途：super 关键字用于引用当前对象的直接父类（超类）。
场景：在子类的方法中，super 用于调用父类的相应方法（当子类重写了父类的方法时）。
在子类的构造函数中，super 用于调用父类的构造函数（必须是构造函数的第一条语句，除非父类没有无参构造函数且子类构造函数也没有参数）。
作用：确保子类能够访问和正确调用父类的属性和方法。
简而言之，this 用于指向当前对象，而 super 用于指向当前对象的直接父类。

二．继承

1. 一个.java 文件可以放多个 class，只有一个主类，主类必须与文件名相同，且一定有 main 方法
2. 子类可以通过 extends 从父类继承，它拥有和父类相同的方法，属性，以及方法实现的功能，和 this 一样，调用构造方法 super()时，要放在构造方法的第一句
   比如 public Fly( ) { super( ); }
3. 子类的同名方法和父类要有差异，若无差异，则会将父类方法覆盖。由对象决定方法。但同名属性可以和父类的并存。
4. Java 允许方法同名而参数不同，系统会根据参数的情况（堆里的对象）选择调用正确的方法。
5. 被子类方法覆盖的父类方法可以不实现，变成抽象方法。比如
   public abstract void Fly( );
   含有一个或多个抽象方法的类就是抽象类，如 abstract class Bee { ---}
6. final 修饰的类不能被继承；final 修饰的属性是常量；final 修饰的方法不能被覆盖
7. 接口 Interface 是一种方法集，跟抽象类有区别，弥补 Java 单根继承的缺陷，是实现多态的另一种方式，比如
   ```
   public interface Fly{
         public void flying( ) ;}
   public class FlyingFish implements Fly{
         public void flying{
               在水里飞行    //这是 FlyingFish 特有的
             }
   }
   ```
8. Public 最开放，private 最封闭，frinedly 同在一个包内都能访问，
   Protected 除了包内可访问，子类也可以访问。
9. Javadoc 是从程序源代码中抽取类，方法，成员等注释形成一个和源代码配套的 API 帮助文档。
10. 静态方法（比如 main 方法）产生的目的，即不依赖对象而直接使用一些方法。它可以直接使用。在其他方法中可以自由调用静态方法，但在静态方法中不能自由使用非静态的对象或属性。
    在 Java 中，静态方法（用 static 关键字修饰的方法）属于类本身，但是非静态变量（没有用 static 关键字修饰的变量）属于类的实例（对象）。因此，静态方法不能直接访问非静态变量，因为非静态变量依赖于特定的对象实例。比如
    ```
    public class jishu{
       int a=5;
       public static void main(String[ ] args) {
          System.out.println(a); }
    }
    ```
    这段代码运行后报错，无法从静态上下文中引用非静态 变量 a
    解决方法： static int a = 5; // 将变量 a 声明为静态变量
    或者，在 main 方法中创建 Jishu 类的一个实例，并通过这个实例访问变量 a
    ```
     Jishu jishu = new Jishu(); // 创建 Jishu 类的一个实例
      System.out.println(jishu.a); // 通过实例访问变量 a
    ```

如果变量 a 的值对于所有 Jishu 类的实例都是相同的，并且您希望能够在不创建对象的情况下访问它，那么第一种方法（将 a 声明为静态变量）是合适的。如果 a 的值可以（或应该）因 Jishu 类的不同实例而异，那么第二种方法（通过对象实例访问 a）是更合适的。

11. 用 final 修饰的属性，变量会放在常量池中，不允许用户改变。
    public static final int INT1 = 9;
    常量的命名约定主要是要求它们使用全大写字母，而下划线是可选的，用于增强可读性，特别是在常量名称包含多个单词。
12. Java 里常量的名称有大写字母和_组成数组可以当做对象来处理，分为声明和初始化两个部分。
    声明：int [ ] arr;（数组句柄为 arr，每个单元存储类型为 int 类型）
    初始化：arr =new int [5];（给数组赋 5 个单元的存储空间，每个单元存储 int 类型，初始值为 0）
13. 字符串 String 是常用的对象，也是一种数组。声明与初始化如下
    String str1=new String(“hello”);
    对于 String str2 = "hello";
    String str3 = "hello";
    System.out.println(str2 == str3); // 输出 true
    System.out.println(str2.equals(str3)); // 输出 true
    str2 是一个字符串变量，被初始化为字面量 "hello"。字符串字面量在 Java 中会被自动放入字符串常量池中，所以 str2 引用的是常量池中的 "hello"。str3 也将引用字符串常量池中的 "hello"。str2 和 str3 是相等的，因为它们都引用字符串常量池中的同一个 "hello" 对象。
    经常讨论的问题
    If(str1==str2) //判断两个对象句柄是否相同
    If(str1.equals(str2)) //两个对象的内容是否相同
14. 我们经常会在一段代码中重复使用一些变量，如：int i，在 Java 中，{…}为一个作用域(scope)，在这个作用域中不能再次定义同名变量。在**同一个作用域**内，确实不能再次声明一个同名的变量。但是，在不同的作用域内，是可以声明同名变量的，并且这些变量是彼此独立的。

```
 int[] arr = new int[]{3, 5, 2, 7, 4, 6, 9, 1};
 int y = 0;
for (int i = 0; i < arr.length; i++) { // 使用 arr.length 而不是硬编码的 8，这样更安全
   y = arr[i];  // 在每次循环迭代中，将数组当前索引 i 处的值赋给变量 y  不是申明操作
                若把这一行改为 int y=arr[i]，会报错，因为前面已经被申明了
   System.out.println("Num: " + y);
}
```

15. 命名规范
    类的命名：一般是名词，每个单词首字母大写，如：Bee, BeeFarming；
    方法命名：一般是动词或动宾结构，第一个字母小写，后每个单词首字母大写，如：flying(), pickHoney()；
    变量命名：同方法命名，最好是类型缩写+含义名词，如：intVolumn, fltPecent；

三．多态

1. 继承还可以支持上溯，上溯是一种非常有趣的想法，将子类的对象赋值给父类的句柄。虽然是父类的句柄，但是无论属性或方法，都还是子类的实现。Bee e = new Honee();代码如下

```
class Bee {
   int a;
   public void fly() {
      System.out.println("Bee is flying") ;  }
 }
class Honee extends Bee {
   int b;
   public void pickHoney() {
      System.out.println("Honee is picking honey");  }
   // 重写 fly 方法（可选，但展示了多态性）
   @Override
   public void fly() {
      System.out.println("Honee is flying with honey"); }
 }

public class Main {
   public static void main(String[ ] args) {
      Honee h = new Honee(); // 创建 Honee 对象
      h.a = 3;
      h.b = 5; // 设置 Honee 对象的属性
      // 调用 Honee 对象的方法
      h.pickHoney(); // 输出: Honee is picking honey
      h.fly();       // 输出: Honee is flying with honey（如果没有重写 fly 方法，即 Honee 中没有 fly 方法，那么打印父类中的 fly 方法）
      // 将 Honee 对象上溯为 Bee 类型的引用
      Bee e = h;
      // 注意：由于e是Bee类型的引用，它不能直接访问Honee特有的属性b和方法pickHoney()
        e.b = 10; // 编译错误，试图访问 e.b 会导致编译错误
      // 调用 fly 方法时，会调用 Honee 类的 fly 方法（如果重写了），展示了多态性
        e.fly(); // 输出: Honee is flying with honey（如果没有重写 fly 方法，即 Honee 中没有 fly 方法，那么打印父类中的 fly 方法）
      // 但是，由于 e 是 Bee 类型的引用，我们不能直接通过 e 调用 pickHoney()方法
        e.pickHoney(); // 编译错误，试图调用 e.pickHoney()会导致编译错误
      // 如果需要通过 e 来调用 pickHoney()方法，可以使用类型转换（但需要注意类型安全问题）
     if (e instanceof Honee) {
        Honee honee = (Honee) e;
         honee. pickHoney(); // 调用 Honee 特有的方法
        honee. b = 10;     // 安全地访问并设置 Honee 特有的属性 b
       } // if 语句的结束大括号，建议单独放在一行以提高可读性        } }
```

- Bee 类是一个基类，有一个 a 属性和一个 fly 方法。
- Honee 类继承自 Bee 类，并添加了一个 b 属性和一个 pickHoney 方法。它还重写了 fly 方法（这是可选的，但用于展示多态性）。
- 在 Main 类的 main 方法中，我们创建了一个 Honee 对象 h，并设置了它的属性 a 和 b。
- 我们通过 h 引用调用了 pickHoney 和 fly 方法。
- 然后，我们将 h 引用上溯为 Bee 类型的引用 e。由于 e 是 Bee 类型的引用，它不能直接访问 Honee 特有的属性 b 和方法 pickHoney。但是，当我们调用 fly 方法时，由于 e 实际上指向一个 Honee 对象，所以会调用 Honee 类的 fly 方法（如果它被重写了）。
- 最后，我们通过类型检查（instanceof）和类型转换（(Honee) e）来安全地访问 Honee 特有的方法和属性。但是，通常建议在设计时尽量避免频繁使用类型转换，因为这可能会破坏代码的可读性和可维护性。

2. 上溯是为了创造出 Java 语言的多态性
   多态是同一名称的方法或属性，由于所属对象不同，而表现出的不同效果。
   多态的好处，是利用同名方法，构造良好的程序设计模式。
3. 属性的初始化
   一个类中，无论属性写在类的哪个部分，都优先初始化这些属性
   属性初始化完成才会初始化构造方法
4. Static 属性又称为类属性，它可以脱离对象存在，因此初始化在对象创建前（类文件载入时，而不在乎是否构造对象。如果有继承关系，不同类中 Static 属性跟前面一般调用构造方法的顺序一致。有继承关系的类，先完成父类属性的初始化，再开始父类构造方法的调用，在初始化子类相关属性和方法。静态属性只初始化 1 次。
5. 静态方法不存在动态性， 静态方法在载入时就明确了对应关系。

四．面向对象小结

6. 面向对象概念小结：封装，将数据和方法组织在一起，实现与使用隔离
   继承，重用父类的实现，约束子类的职能，通过上溯实现多态
   接口，规范无继承关系的类的行为
7. Java 语言的小结：
   class 用来描述封装（类）
   extends 用来描述类的继承关系
   interface 用来描述规范类方法的接口
   new 用来实例化类的具体对象
   super 是父类对象句柄
   this 是本类对象句柄
8. 一个 Java 文件可以包含多个 class，但只有 public class 的名字必须与文件同名
   main 方法必须在 class{……}内，有 main 方法的 java 文件才能执行
   同一个目录下的 Java 程序相当于在同一个包中
9. 构造函数小结
   缺省可以没有构造函数，构造函数必须与类同名(可以有多个)
   一旦有参数的构造函数被定义，就不存在缺省构造函数的问题
   构造函数没有返回值，因为返回的是对象
   有继承关系的子类构造函数调用隐含了其多级父类的构造函数的调用
   构造函数主要用于数据初始化
10. 静态 static 小结
    static 可以修饰属性、方法和类
    static 属性具有对象共享特征，并最早初始化（在 class 文件被载入时初始化，具体在构造函数之前），而且仅初始化 1 次
    static 方法可以脱离对象直接调用，但类方法内部不能使用外面的非静态属性
11. 初始化小结
    普通类初始化顺序：
    .class 文件载入时：静态属性；
    调用构造函数时：普通属性，然后是构造函数
    父子类初始化顺序：
    子类.class 文件载入时会触发父类.class 文件的载入，第一波是全部静态属性
    调用子类构造函数会触发调用父类构造函数，先是父类属性、构造函数，再是子类属性、构造函数
12. 动态绑定小结
    方法重载(overload)：根据输入参数不同自动调用方法
    方法覆盖(override)：根据对象不同而调用同名方法的不同实现
13. 控制设计小结
    对属性和方法的访问控制修饰符：public、protected、friendly、private
    对属性和方法的设计控制：抽象类 abstract
    对常量属性和方法的设计控制：接口 interface
    对内容变化的控制：final

14. JVM、JDK、JRE 是 Java 技术体系中的关键组件，它们各自承担着不同的角色和功能。以下是关于这三者的详细解释：

    JVM（Java Virtual Machine，Java 虚拟机）

    定义：JVM 是一种用于计算设备的规范，是一个虚构出来的计算机，它通过在实际的计算机上仿真模拟各种计算机功能来实现。

    功能：JVM 是 Java 实现跨平台的核心部分。它屏蔽了与具体操作系统平台相关的信息，使得 Java 程序只需生成在 Java 虚拟机上运行的目标代码（字节码），就可以在多种平台上不加修改地运行。此外，JVM 还负责管理内存和垃圾回收，确保 Java 程序的稳定性和安全性。

    JDK（Java Development Kit，Java 开发工具包）

    定义：JDK 是针对 Java 开发者的产品，是整个 Java 开发的核心。

    功能：JDK 提供了构建在 Java 平台上发布的应用程序、applet 和组件的开发环境。它包含了完整的 JRE（Java Runtime Environment，Java 运行时环境），以及供开发者使用的各种 Java 工具和 Java 基础类库。这些工具包括编译器（javac）、解释器（java）、调试器（jdb）等，而 Java 基础类库则包含了大量的 API，供开发者在开发过程中使用。

    JRE（Java Runtime Environment，Java 运行时环境）

    定义：JRE 是运行 Java 程序所必需的环境的集合。

    功能：JRE 包含了 JVM 标准实现及 Java 核心类库。JVM 是 JRE 的核心组件，它提供了一个虚拟的计算机环境，用于执行 Java 字节码。而 Java 核心类库则包含了各种常用的功能和 API，如网络通信、文件操作、数据库连接等。这些类库使得开发者可以更加高效地构建 Java 应用程序。

    三者之间的关系

    JDK 包含 JRE：JDK 是 Java 开发的完整解决方案，它包含了 JRE，使得开发者可以在开发过程中直接运行和测试 Java 程序。

    JRE 包含 JVM：JRE 是运行 Java 程序所必需的环境，它包含了 JVM 和 Java 核心类库，使得 Java 程序可以在不同的操作系统上运行。

    综上所述，JVM、JDK 和 JRE 共同构成了 Java 技术体系的核心部分，它们各自承担着不同的角色和功能，共同支持着 Java 语言的跨平台特性和高效开发。

15. **Java 语言的特点**可简洁地概括为以下几点：

    简单性：去除了指针等复杂特性，易于学习和使用。

    跨平台性：通过 JVM 实现“一次编写，到处运行”，适用于多种操作系统。

    面向对象：支持类、继承、封装和多态，促进模块化、可维护性和扩展性。

    安全性：提供字节码校验、安全管理器等机制，保护程序免受恶意攻击。

    多线程性：支持并发编程，提高程序响应速度和处理能力。

    分布式：易于实现网络通信和分布式计算。

    高性能：尽管是解释型语言，但通过优化和即时编译技术实现高效运行。

    这些特点使得 Java 成为企业级应用、移动开发、大数据处理等领域的首选编程语言。

16. Java 和 C 语言的区别主要体现在以下几个方面，突出重点如下：

    设计思路与编程范式：Java 是面向对象的高级编程语言，强调对象的封装、继承和多态，使得代码更加模块化、易于维护。C 语言是面向过程的低级编程语言，通过函数和过程来组织代码，更接近于计算机硬件的操作方式。

    跨平台性：Java 具有出色的跨平台性，得益于其 Java 虚拟机（JVM）的架构，使得

Java 程序可以在任何安装了 JVM 的平台上运行。C 语言则与具体的操作系统和硬件平台紧密相关，不具有跨平台性，需要在不同的平台上进行编译和链接。

内存管理：Java 提供了自动的内存管理机制，包括垃圾回收机制，减轻了程序员的内存管理负担。C 语言则需要程序员手动管理内存，包括分配和释放内存，这增加了程序员的负担，同时也容易出现内存泄漏等问题。

安全性：Java 在设计时考虑了安全性，提供了多种安全机制，如字节码校验、安全管理器等，使得 Java 程序在运行时更加安全。C 语言则相对较为底层，没有提供类似的安全机制，需要程序员自己注意安全问题。

执行效率：由于 C 语言更接近于计算机硬件，且没有 Java 那样的高级抽象和自动管理机制，因此 C 语言的执行效率通常比 Java 更高。然而，随着 Java 虚拟机（JVM）的不断优化和即时编译（JIT）技术的引入，Java 程序的执行效率也在不断提高。

应用领域：Java 广泛应用于企业级应用开发、移动应用开发、大数据处理等领域，其跨平台性和安全性得到了广泛的认可。C 语言则主要用于系统级编程、嵌入式系统开发、驱动程序开发等领域，其高效性和对硬件的直接控制能力得到了充分的利用。

一句话简单表述就是，java 在代码与操作系统中引入了 JVM。（或者，Java 是面向对象的、跨平台的高级编程语言，注重安全性和内存自动管理；而 C 语言是面向过程的、平台差异化的低级编程语言，需要手动管理内存，执行效率高。）

17. 安装完 JDK 后，我们仅需
    Path：在末尾加入“;……/bin”，用来指明 java 指令路径
    Classpath（1.5 版以后无需配置），JVM 通过 classpath 加载所需的类
    set classpath=.;……tools.jar;……dt.jar
18. **java 程序的编译与运行简洁步骤如下：**
    Path 设置(编译)：使用 javac xxx.java 命令将 Java 源文件编译成字节码，生成 xxx.class 文件。找到 javac.exe，再找到 Java 源文件，再找到 java 的库函数资源，最后编译生成 xxx.class 文件。
    ClassPath 设置(运行)：使用 java xxx 命令（无需.class 后缀）启动 Java 虚拟机（JVM），加载并执行编译后的字节码。找到 java.exe，再找到 xxx.class 文件，再将它载入到虚拟机，再找到 java 的库函数资源并开始执行。
    这两个步骤分别由 Java 编译器（javac）和 Java 解释器（通过 JVM 实现）完成。
19. Java 程序代码存储在“.java”文件中。javac 是编译指令，java 是执行指令。
20. main(String[] args)中的 args 是什么东西？
    首先是个输入参数
    参数类型是 String 类型的数组
    String 是“字符串”类型，后面会经常使用

代码 1，武将排名

```
public class Test {
    private ArrayList<People> peoples = new ArrayList<People>();
    public void read() {
        // [A] 从 data 文件夹里的 originalData.txt 读入数据,生成 people 对象,加入到 ArrayList
        Scanner scanner = new Scanner(new File("data/originalData.txt"));
```

```
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            String[] data = line.split(",");
            String name = data[0];
            int zhihui = Integer.parseInt(data[1]);
            int gongji = Integer.parseInt(data[2]);
            int fangyu = Integer.parseInt(data[3]);
            People p = new People(name, zhihui, gongji, fangyu);
            peoples.add(p)  ;   }
        scanner.close();
    }
    // [B] 按智慧+攻击+防御总和进行排序,将序号写入 people 对象中
  public void setRank() {
    // 创建一个临时列表来存储 people 对象及其对应的总和
         List<People> tempPeoples = new ArrayList<>(peoples.size());
         for (People p : peoples) {
        int sum = p.getZhihui() + p.getGongji() + p.getFangyu();
        p.setTotalSum(sum); // 假设 People 类中有一个字段和方法来存储这个总和
        tempPeoples.add(p);
    }
    // 根据总和对临时列表进行排序
          Collections.sort(tempPeoples, new Comparator<People>() {
        @Override
        public int compare(People p1, People p2) {
            return Integer.compare(p2.getTotalSum(), p1.getTotalSum()); // 降序排序
        }
    });   /*或者用下面语句
        if (wujiang != null && wujiang.length > 0) {
            Arrays.sort(wujiang, (p1, p2) -> {
                return Integer.compare(p2.getTotal(), p1.getTotal());
            }); */

    // 将排序后的列表赋值回 peoples，并设置排名
         peoples = tempPeoples;
         for (int i = 0; i < peoples.size(); i++) {
        peoples.get(i).setRank(i + 1);    }
}
    // [C] 按照 people 中的 rank 值对 peoples 进行排序
  public void ranking() {
                Collections.sort(peoples, new Comparator<People>() {
                @Override
                public int compare(People p1, People p2) {
                                return Integer.compare(p1.getRank(), p2.getRank()); // 升序排序
                } } }
```

```
 public void listRank( ) {
        Iterator<People> it = peoples.iterator();
        while (it.hasNext()) {
            People p = it.next();
            System.out.println("三国排名[" + p.getRank() + "]:" + p.getName() + "智慧(" + p.getZhihui() + ");
攻击(" + p.getGongji() + "),防御(" + p.getFangyu() + ")");
        }   }
 public static void main(String[] args) {
        Test t = new Test();
        t.read();
        t.setRank();
        t.ranking();
        t.listRank();   }
}
class People {
    private String name;
    private int zhihui;
    private int gongji;
    private int fangyu;
    private int rank;
    public People(String name, int zhihui, int gongji, int fangyu) {
        this.name = name;
        this.zhihui = zhihui;
        this.gongji = gongji;
        this.fangyu = fangyu;    }
    // Getter 和 Setter 方法
       public void setName(String name) {
         this.name = name;  }
      public String getName() {
        return this.name;
 }
    public int getZhihui() {
        return this.zhihui;    }
    public void setZhihui(int zhihui) {
        this.zhihui = zhihui;
  }
    public int getGongji() {
        return this.gongji; }
    public void setGongji(int gongji) {
        this.gongji = gongji;
    }
    public int getFangyu() {
        return this. fangyu;      }
    public void setFangyu(int fangyu) {
```

```
        this.fangyu = fangyu;
    }
    public void setRank(int rank) {
        this.rank = rank;    }
    public int getRank() {
        return this.rank;  }
    public int getTotalSum() {
       int  totalSum= this.zhihui + this.gongji + this.fangyu;
        return totalSum;   }
}
```

代码 2，生日

实现 BirthDay 类的构造函数、equals、toString 和 compareTo 方法

根据提供的文件内容，BirthDay类是一个表示出生日期的类，它实现了 Comparable<BirthDay>接口。以下是需要实现的方法：

1. **构造函数**：需要接收年、月、日作为参数，并初始化类的成员变量。
2. **equals 方法**：需要比较两个 BirthDay 对象是否相等，即年、月、日是否都相等。
3. **toString 方法**：需要返回年、月、日的字符串表示。
4. **compareTo 方法**：需要比较两个 BirthDay 对象的大小，根据年、月、日的顺序。

```
class BirthDay implements Comparable<BirthDay> {
    private int year; // 年
    private int month; // 月
    private int day; // 日
    // 构造函数
    public BirthDay(int year, int month, int day) {
        this.year = year;
        this.month = month;
        this.day = day;  }
    // 获得年
    public int getYear() {
        return year;  }
    // 获得月
    public int getMonth() {
        return month;   }
    // 获得日
    public int getDay() {
        return day;      }

    // equals 方法
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
```

```
    if (obj == null || getClass() != obj.getClass()) return false;
    BirthDay birthDay = (BirthDay) obj;
    return year == birthDay.year   &&    month == birthDay.month &&  day == birthDay.day ;    }
  // toString 方法
  @Override
  public String toString() {
    return year + "-" + month + "-" + day;  }
  // compareTo 方法
  @Override
  public int compareTo(BirthDay o) {
    if (this.year != o.year) {
      return this.year - o.year;
    } else if (this.month != o.month) {
      return this.month - o.month;
    } else {
      return this.day - o.day;
    }   } }
```

实现 Person 类的构造函数、equals、toString 和 compareTo 方法

Person 类包含一个 BirthDay 对象和一个 id 字符串，并且实现了 Comparable<Person>接口。以下是需要实现的方法：

1. **构造函数**：需要接收 id 和 BirthDay 对象作为参数，并初始化类的成员变量。
2. **equals 方法**：需要比较两个 Person 对象是否相等，即 id 和出生日期是否都相等。
3. **toString 方法**：需要返回 id 和出生日期的字符串表示。
4. **compareTo 方法**：需要根据出生日期比较两个 Person 对象的大小。

```
class Person implements Comparable<Person> {
  private BirthDay birthDay; // 出生日期
  private String id; // 身份证号
  // 构造函数
  public Person(String id, BirthDay birthDay) {
    this.id = id;
    this.birthDay = birthDay;
  }
  // 获得 id
  public String getId( ) {
    return id;
  }
  // equals 方法
  @Override
  public boolean equals(Object obj) {
    if (this == obj)  return true; //这个语句检查调用 equals 方法的对象（this）是否与传入的参数 obj
```

相同。如果是，意味着它们是同一个对象的引用，因此它们是相等的，方法返回 true。

- this：指向当前的 Person 对象实例。
- obj：是传入 equals 方法的参数，也是一个 Object 类型的对象，预期是与当前对象比较的对象。

所以，this == obj 比较的是两个对象的引用是否相同，如果是，它们指向的是堆内存中的同一个位置，因此它们是相等的。这是一种快速的检查，用于处理两个引用指向同一对象的常见情况

```
        if (obj == null || getClass() != obj.getClass())   return false;
        Person person = (Person) obj;
        return id.equals(person.id) && birthDay.equals(person.birthDay);
    }
    // toString 方法
    @Override
    public String toString() {
        return "ID: " + id + ", BirthDay: " + birthDay.toString();
    }

    // compareTo 方法
    @Override
    public int compareTo(Person o) {
        return this.birthDay.compareTo(o.birthDay);
    }
}
```

问题 3

```
class A {
    public void method1( ) {
        System.out.println("A's method1");
    }
}

class B extends A {
    public void method1( ) {
        System.out.println("B's method1");
    }

    public static void method1(long n) {
        System.out.println("B's static method1");
    }
    public void method2( ) {
        System.out.println("B's method2");
    }
}
```

基于以上类的定义，实例化了以下对象。

```
A o1 = new A( );
A o2 = new B( );
```

同时通过这些语句调用了对象的不同方法，分析哪些方法能正常运行以及原因。

1. `o1.method1();` // ①
   - 正常运行，输出 "A's method1"，因为`o1`是 A 类的实例，它调用了 A 类中的`method1`方法。

2. `o2.method1();` // ②
   - 正常运行，输出 "B's method1"，因为`o2`虽然是 A 类的引用，但它实际上是 B 类的实例，所以调用的是 B 类中的`method1`方法。

3. `o1.method1(1);` // ③
   - 编译错误，因为 A 类中没有接受任何参数的`method1`方法。

4. `o2.method1(1);` // ④
   - 编译错误，因为 B 类中没有接受 int 类型参数的`method1`方法。

5. `o1.method1(1L);` // ⑤
   - 编译错误，同③，A 类中没有接受任何参数的`method1`方法。

6. `o2.method1(1L);` // ⑥
   - 正常运行，输出 "B's static method1"，因为`o2`是 B 类的实例，它调用了 B 类中的静态方法`method1(long n)`。

7. `((B)o1).method2();` // ⑦
   - 运行时异常，因为`o1`实际上是 A 类的实例，没有`method2`方法。

这个语句尝试将`o1`（它是`A`类的实例）强制类型转换为`B`类的对象，然后调用`method2()`方法。这里会发生两件事情：

- **类型转换**：`((B)o1)`这部分是一个显式类型转换（也称为强制类型转换），它试图将`A`类的引用`o1`转换为`B`类的引用。由于`B`类是`A`类的子类，这种向上转型（将子类对象视为父类对象）是隐式且安全的，不需要显式转换，且`o1`实际上引用的是`A`类的对象，所以这个转换在编译时不会报错，但在运行时会失败，因为`A`类中没有`method2`方法。
- **运行时异常**：当尝试调用`method2()`方法时，由于`A`类中不存在这个方法，Java 虚拟机（JVM）会抛出一个`ClassCastException`。这是因为在运行时，JVM 发现`o1`实际上并不是`B`类的实例，而是一个`A`类的实例，因此无法调用`B`类特有的`method2()`方法。

8. `((B)o2).method2();` // ⑧
   - 正常运行，输出 "B's method2"，因为`o2`是 B 类的实例，它调用了 B 类中的`method2`方法。

总结：

- 能正常运行的语句：①、②、⑥、⑧。
- 编译错误的语句：③、④、⑤。因为它们尝试调用不存在的方法。
- 抛出运行时异常的语句：⑦。

问题 4

根据题目描述，我们需要定义一个泛型类`ComputableComparableTuple<T1, T2>`，这个类需要实现`Computable<T1, T2>`和`Comparable<T1, T2>`接口。`T1`和`T2`都必须实现了`Computable`和`Comparable`接口。

```
/*** 泛型接口，可以进行 add 和 subtract 计算的接口 */
interface Computable<T> extends Comparable<T> {
    // 计算二个 T 类型对象的和
    T add(T y);

    // 计算二个 T 类型对象的差
    T subtract(T y);
}
//首先，我们来声明这个类和它的构造函数：
public class ComputableComparableTuple<T1 extends Computable<T1> & Comparable<T1>, T2 extends Computable<T2> & Comparable<T2>>
        implements Computable<T1, T2>, Comparable<T1, T2> {
    private T1 first; // 二元组的第一部分
    private T2 second; // 二元组的第二部分

    // 构造函数
    public ComputableComparableTuple(T1 first, T2 second) {
        this.first = first;
        this.second = second;
    }

    // 省略其他实现代码...
}
```

//接下来，我们需要实现`Computable`接口中的`add`和`subtract`方法，以及`Comparable`接口中的比较方法。由于题目没有提供具体的`Computable`和`Comparable`接口实现细节，我们将假设这些接口有标准的实现方式，以下是`ComputableComparableTuple`类的完整实现：

```
public class ComputableComparableTuple<T1 extends Computable<T1> & Comparable<T1>, T2 extends Computable<T2> & Comparable<T2>>
        implements Computable<T1, T2>, Comparable<T1, T2> {
    private T1 first; // 二元组的第一部分
    private T2 second; // 二元组的第二部分

    // 构造函数
    public ComputableComparableTuple(T1 first, T2 second) {
        this.first = first;
        this.second = second;
    }
```

```
    // 实现 Computable 接口的 add 方法
    @Override
    public T1 add(T2 y) {
        // 假设 T1 和 T2 的 add 方法接受对方类型的参数并返回 T1 类型的结果
        return first.add(y);
    }
    // 实现 Computable 接口的 subtract 方法
    @Override
    public T1 subtract(T2 y) {
        // 假设 T1 和 T2 的 subtract 方法接受对方类型的参数并返回 T1 类型的结果
        return first.subtract(y);
    }

    // 实现 Comparable 接口的 compareTo 方法
    @Override
    public int compareTo(T1 o1, T2 o2) {
        // 比较 first 和 o1，然后比较 second 和 o2
        int result = first.compareTo(o1);
        if (result != 0) {
            return result;
        }
        return second.compareTo(o2);
    }

    // Getter 和 Setter 方法
    public T1 getFirst() {
        return first;
    }
    public void setFirst(T1 first) {
        this.first = first;
    }
    public T2 getSecond() {
        return second;
    }
    public void setSecond(T2 second) {
        this.second = second;
    }
}
```

请注意，这里的实现依赖于`T1`和`T2`的具体实现，特别是它们的`add`、`subtract`和`compareTo`方法。如果`T1`和`T2`的这些方法有不同的签名或行为，你可能需要调整上述代码以适应实际情况。

问题 5

```
public class Main {
    public static void main(String[] args) {
        List<Person> list = new ArrayList<>();
        Person p1= new Person("1",2000,1,25);
        Person p2= new Person("2",2000,1,25);
        Person p3= new Person("3",1998,1,25);
        Person p4= new Person("4",2002,1,25);
        Person[ ]  persons ={ p1,p2p,p3,p4};

        for (Person p : list) {
          if(!list.contains(p))
               list.add(p);
        }
    }
    System.out.println( );
    // 使用 Collections.sort 对 list 进行排序
          Collections.sort(list);
    Iterator<Persons> it = list.iterator;
    while(it.hasNext( )){
        System.out.println( Person.toString );

    }
}
```

打印结果

ID: 1, Year: 1998, Month: 1, Day: 25

ID: 3, Year: 2000, Month: 1, Day: 25

ID: 4, Year: 2002, Month: 1, Day: 25

Collections.sort(list)将会使用对象的自然顺序，调用 CompareTo( )方法，这通常是根据对象的哈希码值，这可能导致不可预测的排序结果。然而，由于 Person 对象的 id、year、month 和 day 属性完全相同，除了 p1 和 p2，它们将被视为相等，因此 p2 不会被添加到列表中。

但若 person 类没有覆盖 equals 方法，那么将直接调用 Object 类中的 equaqls 方法，而此方法仅比较元素的内存地址打印，所以 P1，P2 被判定成不同元素，结果会多出 P2

问题 6

题目要求实现一个方法，该方法可以将一个整数反转。以下是实现代码：

```
public class Test1_1 {
    /**
     * 产生一个反转的十进制整数（高位依次反转到低位）
     * 如：输入 123，返回 321
     * 输入-123，返回-321
     * 输入 0，返回 0
```

```
     * 不考虑数值溢出的情况
     * @param n 十进制整数
     * @return 反转的十进制整数
     */
    public static int reverseInt(int n) {
        int reversed = 0; // 用于存储反转后的整数
        int temp;
        while (n != 0) {
            temp = n % 10; // 获取 n 的最后一位数字
            reversed = reversed * 10 + temp; // 将 temp 添加到 reversed 的末尾
            n /= 10; // 移除 n 的最后一位数字
        }
        return reversed; // 返回反转后的整数
    }

    public static void main(String[] args) {
        System.out.println(reverseInt(123456)); // 应输出 654321
        System.out.println(reverseInt(-123456)); // 应输出-654321
        System.out.println(reverseInt(0)); // 应输出 0
    }
}
```

问题 7

题目要求实现一个字符串解析器，该解析器可以将输入的字符串拆分成单词，并返回一个单词列表。以下是实现代码：

```
class StringParser {
    /**
     * 将输入的字符串拆分成单词
     * 假定字符串里每个单词之间都是以一个空格分开,例如字符串:Hello world
     * @param lineString 输入的字符串
     * @return 拆分得到的单词列表
     */
    public static List<String> parse(String lineString) {
        List<String> list = new ArrayList<>(); // 创建一个列表来存储单词
        if (lineString.isEmpty()) { // 如果输入字符串为空，返回空列表
            return list;
        }
        String[] words = lineString.split(" "); // 使用 split 方法按空格拆分字符串
        for (String word : words) {
            list.add(word); // 将每个单词添加到列表中
        }
        return list; // 返回单词列表
    } }
```

问题 8

补充 A，B，C 部分代码

```
public class TFMath {
    public static void main(String[] args) {
        new TFFrame().launchFrame();
    }
}
    class TFFrame extends Frame {
        TextField num1, num2, result;
        Button btnEqual;

        public void launchFrame() {
            num1 = new TextField(10);
            num2 = new TextField(10);
            result = new TextField(15);
            Label lblPlus = new Label("+");
            btnEqual = new Button("=");

            btnEqual.addActionListener(【A】);

            setLayout(new FlowLayout());
            add(num1);
            add(lblPlus);
            add(num2);
            add(btnEqual);
            add(result);
            pack();
            setVisible(true);
        }

    【B】
}
 【C】
```

根据 2 种方法实现加法计算功能，方法 1 采用内部类实现方式，在【A】【B】两处补充代码

方法 2 采用构造函数传参数方式，在【A】【C】两处补充代码

方法 1

```
        // 【A】处补充代码：使用内部类实例作为事件监听器
        btnEqual.addActionListener(new Monitor());
    // 【B】处补充代码：定义内部类 Monitor
    class Monitor implements ActionListener {
        public void actionPerformed(ActionEvent e) {
            try {
                int n1 = Integer.parseInt(num1.getText());
```

```
            int n2 = Integer.parseInt(num2.getText());
            result.setText("" + (n1 + n2));
        } catch (NumberFormatException ex) {
            System.out.println("请输入有效的数字");
            // 更好的做法是在 UI 中显示错误消息
        }
    }
// 【C】处不需要补充代码，因为所有实现都在 TFFrame 类中完成了。
```

方法 2

```
 // 【A】处补充代码：使用外部 Monitor 类的实例作为事件监听器
    btnEqual.addActionListener(new Monitor(num1, num2, result));
// 【C】处补充代码：定义外部 Monitor 类
class Monitor implements ActionListener {
    private TextField num1, num2, result;

    public Monitor(TextField num1, TextField num2, TextField result) {
        this.num1 = num1;
        this.num2 = num2;
        this.result = result;
    }
    public void actionPerformed(ActionEvent e) {
        try {
            int n1 = Integer.parseInt(num1.getText());
            int n2 = Integer.parseInt(num2.getText());
            result.setText("" + (n1 + n2));
        } catch (NumberFormatException ex) {
            System.out.println("请输入有效的数字");
            // 更好的做法是在 UI 中显示错误消息
        }
    }
}
```